FINAL PROJECT INSTRUCTIONAL GUIDE

P ELEC 1 - Object-Oriented Programming (Python and Django)

Course and Section: _____ Academic Year: _____

Group No.: _____ Project Title: _____

Members: _____

Submission Deadline: _____ Presentation Date: _____

General Instructions: Read all requirements before implementation. Each group shall develop ONE web application from the approved project options. The primary basis for assessment is the correct and meaningful use of Python object-oriented programming (OOP).

Part I. Project Overview and Learning Objectives

This final project requires students to build a small, functional Django web application that applies Python classes and objects to a practical scenario. Django will handle web requests, forms, and page rendering; the application's main calculations and rules must be implemented through Python OOP classes.

Item	Project Requirement
Project type	Group-based web application using Python and Django
Approved choices	Mini POS, Student Grade Computation, ATM Simulator, or Hotel Booking Simulator
Development period	Four (4) weeks from the official project start
Group membership	As approved by the instructor (normally 3-4 members)
Required technologies	Python, Django, HTML, and CSS; JavaScript is optional
Out of scope	Database, authentication, REST APIs, deployment, real payments, and external integrations
Grading	70 points group project output + 30 points individual defense = 100 points

At the end of the project, each student should be able to design meaningful classes; apply encapsulation, inheritance, polymorphism, and exception handling; connect the OOP logic to Django; test normal and invalid conditions; and explain or modify their own code during an individual defense.

Part II. Approved Project Options (Choose ONE)

Groups must select only one option below. Each project is a local educational simulation and must have working forms, visible results, appropriate validation, and functioning Python business logic.

Option A. Mini Point of Sale (POS) System

Develop a small store checkout simulator that accepts item quantities, calculates purchase totals, applies a discount policy, and displays an itemized receipt.

- Display a predefined product catalog with product names and prices.
- Add selected products and positive quantities to a shopping cart; calculate per-item totals and grand total.
- Support at least two customer types with different discount behavior.
- Accept a simulated payment and calculate change; reject payments below the total amount.
- Display a receipt containing item names, quantities, subtotals, discount, total, amount paid, and change.

Suggested classes: Product, CartItem, Cart, Customer, RegularCustomer, DiscountCustomer. Suggested overridden method: calculate_discount().

Option B. Student Grade Computation System

Develop a grading simulator that receives student scores and computes weighted grades using documented assessment rules.

- Accept student information and scores for at least three assessment categories (e.g., quizzes, activities, examinations).
- Show category weights totaling 100%, and compute the weighted final grade accurately.
- Display the final grade and pass/fail remark using a clearly stated threshold.
- Provide at least two grading policies or student/assessment categories with different computation rules.
- Reject missing, nonnumeric, negative, and above-maximum scores; explain errors to the user.

Suggested classes: Student, Assessment, GradeCalculator, RegularCalculator, SpecialCalculator. Suggested overridden method: calculate_grade().

Option C. ATM Simulator

Develop a fictional ATM interface for a sample account, including deposit, withdrawal, balance inquiry, and transaction history.

- Create or select a predefined fictional account with a starting balance.

- Display balance, accept deposit amounts, and process withdrawals.
- Provide at least two account types with different withdrawal behavior or limits.
- Display a simple history of transactions during the active session.
- Reject zero/negative amounts, nonnumeric amounts, and overdrafts; preserve the correct balance after errors.

Suggested classes: Account, SavingsAccount, CheckingAccount, Transaction, ATMService. Suggested overridden method: withdraw().

Option D. Hotel Booking Simulator

Develop a fictional hotel booking and billing application that displays rooms, validates booking details, and computes an estimated bill.

- Display fictional rooms with room numbers, room types, nightly rates, and availability.
- Accept guest name, selected room, and positive number of nights.
- Calculate the total estimated charge and display a booking summary.
- Provide at least two room types with different pricing rules.
- Prevent booking of an occupied room during the active session; reject invalid room selection and stay duration.

Suggested classes: Room, StandardRoom, DeluxeRoom, Guest, Booking, HotelService. Suggested overridden method: calculate_rate().

Part III. Mandatory Python OOP Requirements

All requirements below must be implemented as part of a functioning application workflow. Classes created only to satisfy a checklist, but never called by Django or the program logic, will not be credited.

Concept	Minimum Requirement / Evidence
Classes and Objects	At least four meaningful classes with attributes and methods; objects are created and used in actual workflows.
Encapsulation	Use internal attributes where appropriate, with controlled access and validation through properties or methods.
Inheritance	At least one parent class and two child classes, with a valid "is-a" relationship and shared functionality.
Polymorphism	At least one meaningful overridden method used through a common interface; demonstrate differing results.
Exception Handling	Handle expected invalid inputs and operations and create/use at least one custom exception with a useful message.

Example: In the POS system, RegularCustomer and DiscountCustomer may inherit from Customer and override calculate_discount(). In the Django checkout view, the selected customer object performs the discount calculation. This is a meaningful use of inheritance and polymorphism.

Part IV. Django Integration and Development Rules

The instructor will provide or demonstrate a basic Django starter project. Students are responsible for connecting the Python OOP logic to the web pages and understanding the request-to-response process.

- Use Django URL routes, views, and templates so that users can interact with the selected project through a browser.
- Keep business rules, calculations, and validations primarily in separate Python class modules, rather than hard-coding them in HTML templates or views.
- Use HTML forms for input and display understandable success messages and validation errors.
- A database is NOT required. Predefined data, in-memory objects, and/or Django session data may be used. Document any data reset or persistence limitations.
- Login, registration, REST API endpoints, external APIs, payment processing, and online deployment are NOT required.
- CSS may be used to make the application presentable; JavaScript is optional but shall not replace required Python OOP logic.
- All user data and transactions are fictional. Do not collect real financial or sensitive personal information.

Part V. Recommended Four-Week Development Plan

Week	Tasks and Expected Output
Week 1 - Planning	Select project, finalize required workflows, assign member responsibilities, define validation rules, and prepare initial UML class diagram.
Week 2 - OOP Core	Build and test classes, constructor logic, inheritance, method overriding, encapsulation, and custom exceptions in Python.

Week	Tasks and Expected Output
Week 3 - Django Integration	Connect URLs, views, forms, and templates to the OOP classes; verify correct results and session-based behavior.
Week 4 - Quality and Defense	Run at least six documented test cases, revise documentation and UML, prepare ZIP, rehearse group presentation and individual defense.

Part VI. Documentation and Submission

Submit ONE ZIP archive per group with the following contents:

- Runnable Django source code. Include manage.py, project/app modules, templates, and static files. Exclude virtual environments, cache folders, and secret credentials.
- README.md specifying the project title, group members, Python version, setup commands, run instructions, main workflow, and known limitations.
- UML class diagram showing the key classes, important attributes/methods, and inheritance relationships.
- OOP implementation summary mapping each required concept to the specific Python file, class, and method where it is demonstrated.
- Test report of at least six (6) cases covering normal, invalid, and boundary input; include input, expected result, actual result, and pass/fail.
- Member contribution record specifying assigned responsibilities and completed features.

The final ZIP naming format, submission platform, confirmed deadline, and presentation schedule shall be announced by the instructor. All members must verify that the application runs using the submitted README instructions.

Part VII. Group Presentation and Individual Defense

During the group demonstration, explain the project purpose, show each required feature from input to output, and identify the Python classes responsible for business rules. Demonstrate invalid input handling and describe how Django views call the OOP classes.

Every student will complete an individual defense. Each student must (1) explain their own code and its relationship to other classes, (2) perform an instructor-assigned live change, and (3) run or demonstrate the result of that change. Examples include revising a discount/grade/rate rule, adding validation, or changing an overridden method. Each student is graded separately for this component.

Part VIII. Grading Rubrics

The final grade is based on a 100-point rubric. The group project component is shared by group members; the individual defense component may differ for each student.

A. Group Project Output (70 points)

Criterion	Evidence of Excellent Performance	Points
OOP Design and Application	Classes correctly model the problem; encapsulation, inheritance, and polymorphism are meaningful and integrated into the workflow.	25
Functional Correctness	All required project features operate correctly; calculations, state, and outputs are accurate.	20
Django Integration and Usability	Routes, views, templates, and forms work together; the application is clear and usable.	10
Error Handling and Testing	Custom exception and validations work; six documented tests cover typical, invalid, and boundary cases.	10
Documentation and Submission	ZIP, README, UML class diagram, OOP mapping, test report, and contribution record are complete.	5
TOTAL		70

B. Individual Defense (30 points)

Criterion	Evidence of Excellent Performance	Points
Code Explanation	Explains own classes and methods, object interactions, inheritance/overriding, and request-to-result flow accurately.	12
Live Code Modification	Independently completes the requested change, runs the revised code, and explains the effect.	12
Contribution and Workflow	Identifies own contributions and explains their interaction with the rest of the application.	6
TOTAL		30

C. Rating Guide for Each Criterion

Level	Performance Description	Scoring Range
Excellent	Complete, correct, independently demonstrated, and clearly explained.	90-100%
Proficient	Mostly complete/correct; minor errors or explanation gaps.	75-89%
Developing	Partially working or insufficiently explained; significant gaps.	50-74%
Beginning	Missing, nonfunctional, or unable to demonstrate understanding.	0-49%

Scoring: Instructors assign whole-number points for each criterion based on observed evidence. Final Project Score = Group Project Score (maximum 70) + Individual Defense Score (maximum 30).

Part IX. Test Case and Defense Preparation Guide

The test report must contain at least six cases. Select cases that match your chosen application and include the exact user input, expected output, actual output, result (Pass/Fail), and a short explanation for failures. The examples below are guides; choose your own appropriate values and stated rules.

Test Type	Example Scenario / Expected Behavior
Normal operation	POS: valid cart and sufficient payment produce the correct receipt and change.
Boundary input	Grade system: check values at the accepted limits, such as 0 and 100.
Invalid input	ATM: reject a negative deposit and keep the balance unchanged.
Business rule	Hotel: reject a booking request for an occupied room within the session.
Polymorphism	Run the same overridden method for two child-class objects; verify distinct results.
Exception handling	Trigger the custom exception and confirm a readable message appears in the browser.

For defense preparation, be ready to locate the file containing each core class; distinguish the parent and child responsibilities; explain why overriding is needed; trace one HTML form submission through the Django view to an OOP method; and show how a rejected operation avoids invalid application state.

Part X. Academic Integrity and Important Reminders

- Every group member must understand the application's main OOP design and workflow, not only the portion they wrote.
- References, documentation, and permitted libraries may be consulted; students must still be able to explain and modify submitted code independently.
- A static website without functioning Python OOP logic does not meet the project requirements.
- Demonstrate input validation and explain how the application handles invalid requests or operations.
- Be prepared to run the project locally without paid services or internet-dependent functionality.

Note: The instructor may issue written clarifications regarding team membership, starter code, schedules, or administrative submission details. The learning outcomes and grading categories above remain the basis for assessment unless formally revised.

Prepared for: P ELEC 1 - Object-Oriented Programming | BS Information Technology